EN3160 Image Processing and Machine Vision

Assignment 3 – Neural Networks

Name: S.A.D. Priyadarshana

Index: 230502X

GitHub: denethp/CV-Assignment-03

Question 1

[20]:
```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

%matplotlib inline

_ = torch.manual_seed(0)  # fixed seed so my runs are reproducible
```

[21]:
```
# CIFAR-10, normalized to [-1, 1]
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 50

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')
```

[22]:
```
def accuracy(loader, forward):
    # Percentage of correctly classified images in a loader
    correct, total = 0, 0
    with torch.no_grad():
        for inputs, labels in loader:
            x = inputs.view(inputs.shape[0], -1)
            predicted = torch.argmax(forward(x), dim=1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return 100 * correct / total
```

Baseline: single dense layer with MSE loss

First, I trained the given network without changes so I have a result to compare with. With batch size N, input size $d = 3072$ and $k = 10$ classes, it computes

$$\hat{y} = xw + \mathbf{1}_N b^\top, \qquad x \in \mathbb{R}^{N \times d}, \quad w \in \mathbb{R}^{d \times k}, \quad b \in \mathbb{R}^k$$

and is trained with the MSE loss between $\hat{y}$ and the one-hot labels y.

[23]:
```
Din = 3 * 32 * 32  # flattened CIFAR-10 image
K = 10             # number of classes
std = 1e-5

w = torch.randn(Din, K) * std
b = torch.zeros(K)

iterations = 20
lr = 2e-6
lr_decay = 0.9
reg = 0
loss_history = []

for t in range(iterations):
    running_loss = 0.0
    for inputs, labels in trainloader:
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)
        y_train_onehot = nn.functional.one_hot(labels, K).float()

        # Forward pass
        y_pred = x_train.mm(w) + b

        # MSE loss with L2 regularization
        loss = (1 / Ntr) * torch.sum((y_pred - y_train_onehot) ** 2) + reg * torch.sum(w ** 2)
        loss_history.append(loss.item())
        running_loss += loss.item()

        # Backpropagation
        dy_pred = (2.0 / Ntr) * (y_pred - y_train_onehot)
        dw = x_train.t().mm(dy_pred) + reg * w
        db = dy_pred.sum(dim=0)

        w -= lr * dw
        b -= lr * db

    print(f'Epoch {t + 1}/{iterations}, Loss: {running_loss / len(trainloader):.4f}')
    lr *= lr_decay
```

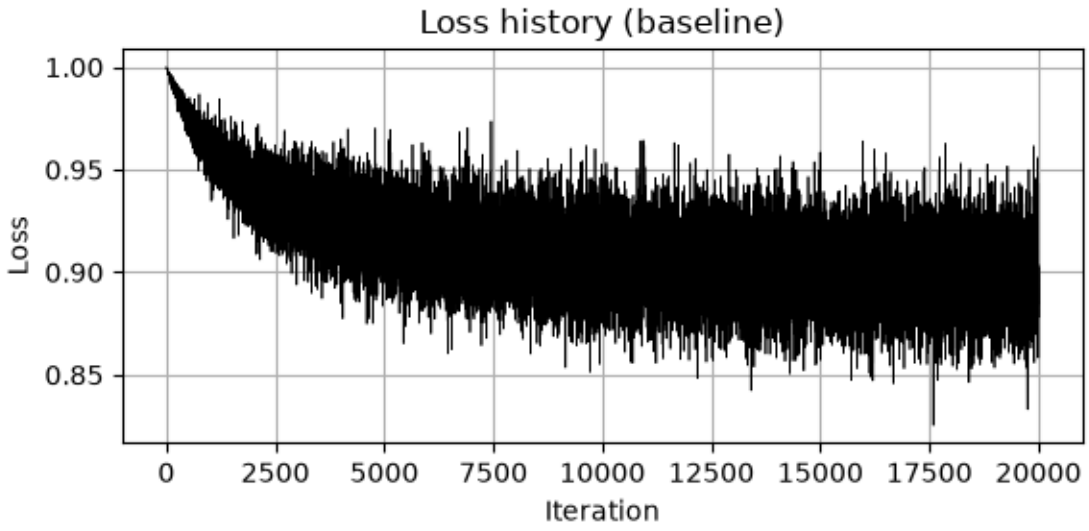
```
Epoch 1/20, Loss: 0.9769
Epoch 2/20, Loss: 0.9498
Epoch 3/20, Loss: 0.9361
Epoch 4/20, Loss: 0.9275
Epoch 5/20, Loss: 0.9216
Epoch 6/20, Loss: 0.9172
Epoch 7/20, Loss: 0.9138
Epoch 8/20, Loss: 0.9111
Epoch 9/20, Loss: 0.9088
Epoch 10/20, Loss: 0.9070
Epoch 11/20, Loss: 0.9054
Epoch 12/20, Loss: 0.9041
Epoch 13/20, Loss: 0.9030
Epoch 14/20, Loss: 0.9020
```

```
Epoch 15/20, Loss: 0.9011
Epoch 16/20, Loss: 0.9004
Epoch 17/20, Loss: 0.8997
Epoch 18/20, Loss: 0.8991
Epoch 19/20, Loss: 0.8986
Epoch 20/20, Loss: 0.8982
```

[24]:
```
baseline_train_acc = accuracy(trainloader, lambda x: x.mm(w) + b)
baseline_test_acc = accuracy(testloader, lambda x: x.mm(w) + b)
print(f'Training accuracy: {baseline_train_acc:.2f}%')
print(f'Test accuracy: {baseline_test_acc:.2f}%')

plt.figure(figsize=(6, 3))
plt.plot(loss_history, color='k', linewidth=0.5)
plt.title('Loss history (baseline)')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Training accuracy: 32.22%
Test accuracy: 32.44%
```



(a) Adding a middle layer with 100 nodes and sigmoid activation

I added a hidden layer with $H = 100$ nodes and the sigmoid $\sigma(t) = \frac{1}{1+e^{-t}}$:

$$h = \sigma(xw_1 + \mathbf{1}_N b_1^\top), \qquad \hat{y} = hw_2 + \mathbf{1}_N b_2^\top$$

with $w_1 \in \mathbb{R}^{d\times H}$ and $w_2 \in \mathbb{R}^{H\times k}$. The gradients are still computed by hand. With $g \equiv \frac{\partial L}{\partial \hat{y}}$ (called `dy_pred` in the code), the backward pass is

$$\frac{\partial L}{\partial w_2} = h^\top g, \qquad \frac{\partial L}{\partial b_2} = \mathbf{1}_N^\top g, \qquad g_h = (g\, w_2^\top) \odot h \odot (1-h)$$

$$\frac{\partial L}{\partial w_1} = x^\top g_h, \qquad \frac{\partial L}{\partial b_1} = \mathbf{1}_N^\top g_h$$

where $h \odot (1-h)$ is the derivative of the sigmoid. Each parameter is then updated with $w \leftarrow w - \eta \frac{\partial L}{\partial w}$.

I initialized the weights with standard deviation $1/\sqrt{\text{inputs}}$ instead of 10^{-5}, because with such small weights every hidden node outputs about 0.5 and the network learns very slowly. I also moved the data to the GPU to make training faster.

[25]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

def to_device_tensors(dataset):
    # Same preprocessing as the transform above: [0, 1] -> [-1, 1], then flatten (C, H, W)
    x = torch.tensor(dataset.data, dtype=torch.float32).permute(0, 3, 1, 2) / 255.0
    x = ((x - 0.5) / 0.5).reshape(len(dataset), -1)
    y = torch.tensor(dataset.targets)
    return x.to(device), y.to(device)

X_train, Y_train = to_device_tensors(trainset)
X_test, Y_test = to_device_tensors(testset)
print('Train:', tuple(X_train.shape), ' Test:', tuple(X_test.shape))
```

```
Device: cuda
Train: (50000, 3072)  Test: (10000, 3072)
```

[26]:
```
Din = 3 * 32 * 32
H = 100  # hidden nodes
K = 10

def init_params(seed=0):
    g = torch.Generator(device=device).manual_seed(seed)
    w1 = torch.randn(Din, H, device=device, generator=g) / Din ** 0.5
    b1 = torch.zeros(H, device=device)
    w2 = torch.randn(H, K, device=device, generator=g) / H ** 0.5
    b2 = torch.zeros(K, device=device)
    return [w1, b1, w2, b2]

def forward(x, params):
    w1, b1, w2, b2 = params
    h = torch.sigmoid(x.mm(w1) + b1)  # hidden layer
    y_pred = h.mm(w2) + b2            # output scores
    return y_pred, h

def evaluate(X, Y, params):
    with torch.no_grad():
        predicted = torch.argmax(forward(X, params)[0], dim=1)
    return 100 * (predicted == Y).float().mean().item()

def train(loss_fn, epochs=10, lr=0.05, lr_decay=0.9, reg=0.0, seed=0):
    # Mini-batch gradient descent with manually computed gradients
    params = init_params(seed)
    w1, b1, w2, b2 = params
```

```
    g = torch.Generator(device=device).manual_seed(seed)
    N = X_train.shape[0]
    loss_history, epoch_loss = [], []

    for epoch in range(epochs):
        perm = torch.randperm(N, device=device, generator=g)
        running_loss = 0.0
        for i in range(0, N, batch_size):
            idx = perm[i:i + batch_size]
            x = X_train[idx]
            y_onehot = nn.functional.one_hot(Y_train[idx], K).float()

            # Forward pass
            y_pred, h = forward(x, params)
            loss, dy_pred = loss_fn(y_pred, y_onehot)
            loss = loss + reg * (torch.sum(w1 ** 2) + torch.sum(w2 ** 2))

            # Backpropagation
            dw2 = h.t().mm(dy_pred) + 2 * reg * w2
            db2 = dy_pred.sum(dim=0)
            dz1 = dy_pred.mm(w2.t()) * h * (1 - h)  # through the sigmoid
            dw1 = x.t().mm(dz1) + 2 * reg * w1
            db1 = dz1.sum(dim=0)

            # Parameter update
            w1 -= lr * dw1
            b1 -= lr * db1
            w2 -= lr * dw2
            b2 -= lr * db2

            loss_history.append(loss.item())
            running_loss += loss.item()

        epoch_loss.append(running_loss / (N // batch_size))
        print(f'Epoch {epoch + 1}/{epochs}, Loss: {epoch_loss[-1]:.4f}')
        lr *= lr_decay

    return params, loss_history, epoch_loss
```

To see the effect of the hidden layer alone, I first trained it with the same MSE loss for 10 epochs, where $L = \frac{1}{N}\|\hat{y} - y\|^2$ and $g = \frac{2}{N}(\hat{y} - y)$.

[27]:
```
def mse_loss(y_pred, y_onehot):
    n = y_pred.shape[0]
    loss = torch.sum((y_pred - y_onehot) ** 2) / n
    dy_pred = 2.0 * (y_pred - y_onehot) / n
    return loss, dy_pred

params_mse, loss_history_mse, epoch_loss_mse = train(mse_loss, epochs=10, lr=0.05)

mse_train_acc = evaluate(X_train, Y_train, params_mse)
mse_test_acc = evaluate(X_test, Y_test, params_mse)
print(f'Training accuracy: {mse_train_acc:.2f}%')
print(f'Test accuracy: {mse_test_acc:.2f}%')
```

```
Epoch 1/10, Loss: 2.0829
Epoch 2/10, Loss: 0.8287
Epoch 3/10, Loss: 0.7952
```

```
Epoch 4/10, Loss: 0.7732
Epoch 5/10, Loss: 0.7589
Epoch 6/10, Loss: 0.7472
Epoch 7/10, Loss: 0.7385
Epoch 8/10, Loss: 0.7321
Epoch 9/10, Loss: 0.7258
Epoch 10/10, Loss: 0.7208
Training accuracy: 47.71%
Test accuracy: 44.01%
```

(b) Using the cross-entropy loss

Next, I replaced MSE with the cross-entropy loss. Softmax turns each row of $\hat{y}$ into class probabilities p, and the loss is the average of $-\log$ of the probability of the correct class:

$$p_{ij} = \frac{e^{\hat{y}_{ij}}}{\sum_{l=1}^{k} e^{\hat{y}_{il}}}, \qquad L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{k} y_{ij} \log p_{ij}$$

Its gradient is simply $g = \frac{1}{N}(p - y)$, so only g changes and the rest of the backward pass stays the same. All other settings are the same as the MSE run, so the two losses can be compared fairly.

[28]:
```python
def cross_entropy_loss(y_pred, y_onehot):
    n = y_pred.shape[0]
    log_p = torch.log_softmax(y_pred, dim=1)  # numerically stable log-softmax
    loss = -torch.sum(y_onehot * log_p) / n
    dy_pred = (torch.exp(log_p) - y_onehot) / n
    return loss, dy_pred

params_ce, loss_history_ce, epoch_loss_ce = train(cross_entropy_loss, epochs=10, lr=0.05)

ce_train_acc = evaluate(X_train, Y_train, params_ce)
ce_test_acc = evaluate(X_test, Y_test, params_ce)
print(f'Training accuracy: {ce_train_acc:.2f}%')
print(f'Test accuracy: {ce_test_acc:.2f}%')
```

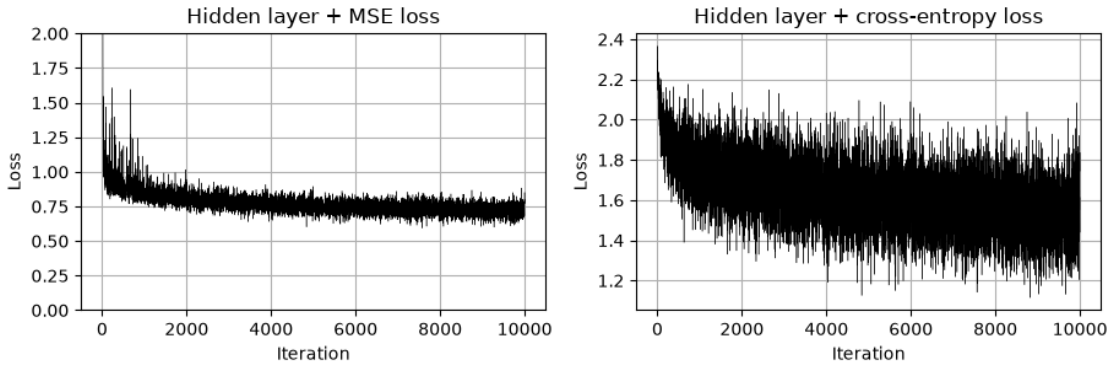
```
Epoch 1/10, Loss: 1.8510
Epoch 2/10, Loss: 1.7234
Epoch 3/10, Loss: 1.6740
Epoch 4/10, Loss: 1.6395
Epoch 5/10, Loss: 1.6123
Epoch 6/10, Loss: 1.5899
Epoch 7/10, Loss: 1.5693
Epoch 8/10, Loss: 1.5524
Epoch 9/10, Loss: 1.5369
Epoch 10/10, Loss: 1.5223
Training accuracy: 48.55%
Test accuracy: 45.94%
```

(c) Results after 10 epochs

The final network (hidden layer + cross-entropy) was trained for 10 epochs above. Below are the loss curves and the accuracies of all three networks.

[29]:
```python
fig, axes = plt.subplots(1, 2, figsize=(9.3, 3.2))
curves = [(loss_history_mse, 'Hidden layer + MSE loss'),
          (loss_history_ce, 'Hidden layer + cross-entropy loss')]
```

```
for ax, (hist, title) in zip(axes, curves):
    ax.plot(hist, color='k', linewidth=0.3)
    ax.set_title(title)
    ax.set_xlabel('Iteration')
    ax.set_ylabel('Loss')
    ax.grid(True)
axes[0].set_ylim(0, 2)  # the first few MSE iterations are much larger
plt.tight_layout()
plt.show()
```



[30]:
```
results = [('Single layer + MSE (20 epochs)', baseline_train_acc, baseline_test_acc),
           ('Hidden layer + MSE (10 epochs)', mse_train_acc, mse_test_acc),
           ('Hidden layer + cross-entropy (10 epochs)', ce_train_acc, ce_test_acc)]

print(f"{'Network':<42}{'Train acc':>11}{'Test acc':>11}")
for name, tr, te in results:
    print(f'{name:<42}{tr:>10.2f}%{te:>10.2f}%')
```

```
Network                                    Train acc   Test acc
Single layer + MSE (20 epochs)                32.22%     32.44%
Hidden layer + MSE (10 epochs)                47.71%     44.01%
Hidden layer + cross-entropy (10 epochs)      48.55%     45.94%
```

Discussion: the hidden layer gave the biggest improvement. With the same MSE loss, the test accuracy rose from 32.44% (single layer, 20 epochs) to 44.01% (10 epochs). The single-layer network is a linear classifier, so it cannot learn complex patterns. Its training and test accuracies were almost the same (32.22% and 32.44%), which means it was limited by its simplicity, not by overfitting. The sigmoid hidden layer makes the network non-linear, so it can fit the data much better.

Changing MSE to cross-entropy raised the test accuracy further to 45.94%. Cross-entropy focuses on making the correct class most probable, while MSE also tries to push every wrong class score to exactly 0, which is not needed for classification. The cross-entropy loss was still decreasing at epoch 10, so more epochs or a larger learning rate would probably improve the accuracy further.

Question 2

[31]:
```
import time
import torch.nn.functional as F
import torch.optim as optim
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# MNIST, normalized with the dataset mean and standard deviation
mnist_train = torchvision.datasets.MNIST(root='./data', train=True, download=True)
mnist_test = torchvision.datasets.MNIST(root='./data', train=False, download=True)

def mnist_to_device(dataset):
    x = dataset.data.float().unsqueeze(1) / 255.0  # (N, 1, 28, 28)
    x = (x - 0.1307) / 0.3081
    return x.to(device), dataset.targets.to(device)

Xm_train, Ym_train = mnist_to_device(mnist_train)
Xm_test, Ym_test = mnist_to_device(mnist_test)
print('Train:', tuple(Xm_train.shape), ' Test:', tuple(Xm_test.shape))
```

```
Train: (60000, 1, 28, 28)  Test: (10000, 1, 28, 28)
```

I wrote one training function for both networks so they are trained in exactly the same way: Adam optimizer (learning rate 10^{-3}), batch size 64, cross-entropy loss and 10 epochs. After each epoch it records the training and test loss and accuracy for the plots.

[32]:
```
def evaluate_model(model, X, Y, batch=1000):
    # Average loss and accuracy (%) in evaluation mode
    model.eval()
    total_loss, correct = 0.0, 0
    with torch.no_grad():
        for i in range(0, len(X), batch):
            out = model(X[i:i + batch])
            total_loss += F.cross_entropy(out, Y[i:i + batch], reduction='sum').item()
            correct += (out.argmax(dim=1) == Y[i:i + batch]).sum().item()
    return total_loss / len(X), 100 * correct / len(X)

def fit(model, epochs=10, lr=1e-3, batch=64, seed=0):
    torch.manual_seed(seed)
    optimizer = optim.Adam(model.parameters(), lr=lr)
    history = {'train_loss': [], 'train_acc': [], 'test_loss': [], 'test_acc': []}
    N = len(Xm_train)

    for epoch in range(epochs):
        model.train()
        start = time.time()
        perm = torch.randperm(N, device=device)
        total_loss, correct = 0.0, 0
        for i in range(0, N, batch):
            idx = perm[i:i + batch]
            x, y = Xm_train[idx], Ym_train[idx]

            out = model(x)
            loss = F.cross_entropy(out, y)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item() * len(y)
            correct += (out.argmax(dim=1) == y).sum().item()

        test_loss, test_acc = evaluate_model(model, Xm_test, Ym_test)
        history['train_loss'].append(total_loss / N)
```

```
        history['train_acc'].append(100 * correct / N)
        history['test_loss'].append(test_loss)
        history['test_acc'].append(test_acc)
        print(f"Epoch {epoch + 1}/{epochs}: "
              f"train loss {history['train_loss'][-1]:.4f}, train acc {history['train_acc'][-1]:.
↪2f}% | "
              f"test loss {test_loss:.4f}, test acc {test_acc:.2f}% ({time.time() - start:.
↪1f}s)")
    return history
```

(a) LeNet-5

I built LeNet-5 as in the original paper (LeCun et al., 1998), with tanh activations and average pooling. MNIST images are 28×28, so I padded the first convolution by 2 pixels to get the same sizes as the original 32×32 input:

- C1: 5×5 convolution, 6 filters, tanh $\rightarrow 6 \times 28 \times 28$
- S2: 2×2 average pooling $\rightarrow 6 \times 14 \times 14$
- C3: 5×5 convolution, 16 filters, tanh $\rightarrow 16 \times 10 \times 10$
- S4: 2×2 average pooling $\rightarrow 16 \times 5 \times 5$
- Fully connected layers: $120 \rightarrow 84 \rightarrow 10$

[33]:
```
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.c1 = nn.Conv2d(1, 6, kernel_size=5, padding=2)
        self.s2 = nn.AvgPool2d(2)
        self.c3 = nn.Conv2d(6, 16, kernel_size=5)
        self.s4 = nn.AvgPool2d(2)
        self.c5 = nn.Linear(16 * 5 * 5, 120)
        self.f6 = nn.Linear(120, 84)
        self.out = nn.Linear(84, 10)

    def forward(self, x):
        x = self.s2(torch.tanh(self.c1(x)))
        x = self.s4(torch.tanh(self.c3(x)))
        x = torch.flatten(x, 1)
        x = torch.tanh(self.c5(x))
        x = torch.tanh(self.f6(x))
        return self.out(x)

torch.manual_seed(0)
lenet = LeNet5().to(device)
print(f'LeNet-5 parameters: {sum(p.numel() for p in lenet.parameters()):,}')

history_lenet = fit(lenet, epochs=10)
print(f"\nLeNet-5 after 10 epochs: training accuracy {history_lenet['train_acc'][-1]:.2f}%, "
      f"test accuracy {history_lenet['test_acc'][-1]:.2f}%")
```

```
LeNet-5 parameters: 61,706
Epoch 1/10: train loss 0.2575, train acc 92.54% | test loss 0.0886, test acc
97.55% (5.2s)
Epoch 2/10: train loss 0.0735, train acc 97.81% | test loss 0.0607, test acc
98.09% (5.1s)
Epoch 3/10: train loss 0.0524, train acc 98.36% | test loss 0.0526, test acc
98.27% (4.9s)
Epoch 4/10: train loss 0.0396, train acc 98.77% | test loss 0.0542, test acc
```

```
98.28% (5.0s)
Epoch 5/10: train loss 0.0334, train acc 98.95% | test loss 0.0458, test acc
98.40% (5.1s)
Epoch 6/10: train loss 0.0277, train acc 99.12% | test loss 0.0463, test acc
98.64% (5.0s)
Epoch 7/10: train loss 0.0242, train acc 99.20% | test loss 0.0380, test acc
98.71% (5.0s)
Epoch 8/10: train loss 0.0205, train acc 99.35% | test loss 0.0495, test acc
98.56% (5.1s)
Epoch 9/10: train loss 0.0169, train acc 99.47% | test loss 0.0522, test acc
98.42% (5.1s)
Epoch 10/10: train loss 0.0148, train acc 99.51% | test loss 0.0402, test acc
98.77% (5.1s)

LeNet-5 after 10 epochs: training accuracy 99.51%, test accuracy 98.77%
```

(b) A modern small CNN with batch normalization and dropout

My modern CNN has three blocks of 3×3 convolution $\rightarrow$ batch normalization $\rightarrow$ ReLU $\rightarrow$ 2×2 max pooling, with 32, 64 and 128 filters. After that comes dropout (0.5), a fully connected layer with 128 units, dropout (0.5) again, and the output layer.

The main differences from LeNet-5:

- ReLU instead of tanh, so gradients do not become very small.
- Batch normalization, which keeps the values in each layer well scaled and makes training faster and more stable.
- Dropout, which randomly turns off half of the units during training to reduce overfitting.
- Max pooling and more filters.

It is trained with the same `fit` function and settings as LeNet-5.

[34]:
```python
class ModernCNN(nn.Module):
    def __init__(self):
        super().__init__()
        def block(c_in, c_out):
            return nn.Sequential(
                nn.Conv2d(c_in, c_out, kernel_size=3, padding=1, bias=False),
                nn.BatchNorm2d(c_out),
                nn.ReLU(),
                nn.MaxPool2d(2))
        self.features = nn.Sequential(block(1, 32), block(32, 64), block(64, 128))
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(0.5),
            nn.Linear(128 * 3 * 3, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 10))

    def forward(self, x):
        return self.classifier(self.features(x))

torch.manual_seed(0)
cnn = ModernCNN().to(device)
print(f'Modern CNN parameters: {sum(p.numel() for p in cnn.parameters()):,}')

history_cnn = fit(cnn, epochs=10)
```

```
print(f"\nModern CNN after 10 epochs: training accuracy {history_cnn['train_acc'][-1]:.2f}%, "
      f"test accuracy {history_cnn['test_acc'][-1]:.2f}%")
```

```
Modern CNN parameters: 241,770
Epoch 1/10: train loss 0.2097, train acc 93.54% | test loss 0.0487, test acc
98.46% (6.1s)
Epoch 2/10: train loss 0.0831, train acc 97.60% | test loss 0.0298, test acc
99.02% (6.1s)
Epoch 3/10: train loss 0.0664, train acc 98.09% | test loss 0.0249, test acc
99.15% (6.7s)
Epoch 4/10: train loss 0.0573, train acc 98.31% | test loss 0.0259, test acc
99.12% (6.5s)
Epoch 5/10: train loss 0.0509, train acc 98.51% | test loss 0.0230, test acc
99.27% (6.3s)
Epoch 6/10: train loss 0.0436, train acc 98.70% | test loss 0.0207, test acc
99.33% (6.5s)
Epoch 7/10: train loss 0.0404, train acc 98.79% | test loss 0.0248, test acc
99.29% (6.4s)
Epoch 8/10: train loss 0.0392, train acc 98.81% | test loss 0.0239, test acc
99.30% (6.7s)
Epoch 9/10: train loss 0.0335, train acc 98.99% | test loss 0.0249, test acc
99.25% (6.4s)
Epoch 10/10: train loss 0.0309, train acc 99.03% | test loss 0.0193, test acc
99.34% (6.5s)

Modern CNN after 10 epochs: training accuracy 99.03%, test accuracy 99.34%
```

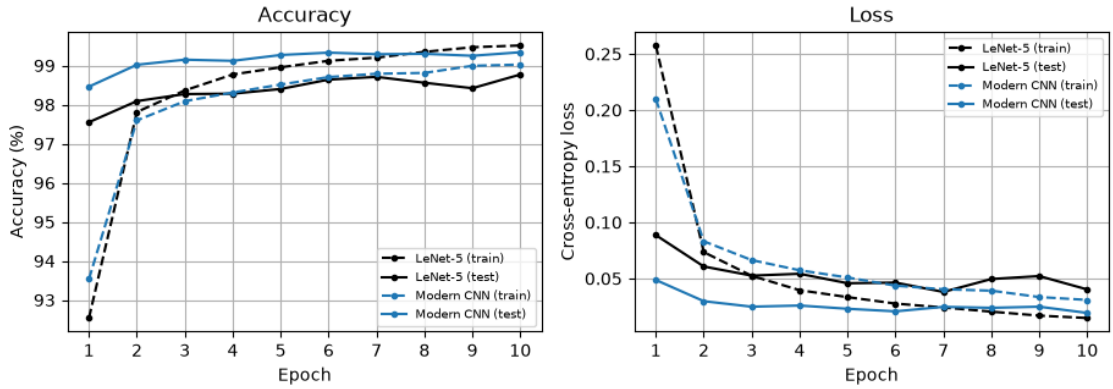
(c) Learning curves and confusion matrices

Accuracy and loss per epoch for both models (dashed = training, solid = test).

[35]:
```
epochs = range(1, 11)
models = [(history_lenet, 'LeNet-5', 'k'), (history_cnn, 'Modern CNN', 'tab:blue')]

fig, axes = plt.subplots(1, 2, figsize=(9.3, 3.4))
for hist, name, color in models:
    axes[0].plot(epochs, hist['train_acc'], '--o', color=color, markersize=3, label=f'{name}␣
 ↪(train)')
    axes[0].plot(epochs, hist['test_acc'], '-o', color=color, markersize=3, label=f'{name}␣
 ↪(test)')
    axes[1].plot(epochs, hist['train_loss'], '--o', color=color, markersize=3, label=f'{name}␣
 ↪(train)')
    axes[1].plot(epochs, hist['test_loss'], '-o', color=color, markersize=3, label=f'{name}␣
 ↪(test)')

axes[0].set_title('Accuracy')
axes[0].set_ylabel('Accuracy (%)')
axes[1].set_title('Loss')
axes[1].set_ylabel('Cross-entropy loss')
for ax in axes:
    ax.set_xlabel('Epoch')
    ax.set_xticks(epochs)
    ax.grid(True)
    ax.legend(fontsize=7)
plt.tight_layout()
plt.show()
```



Confusion matrices on the test set. Row i, column j shows how many images of digit i were predicted as j, so correct predictions are on the diagonal.

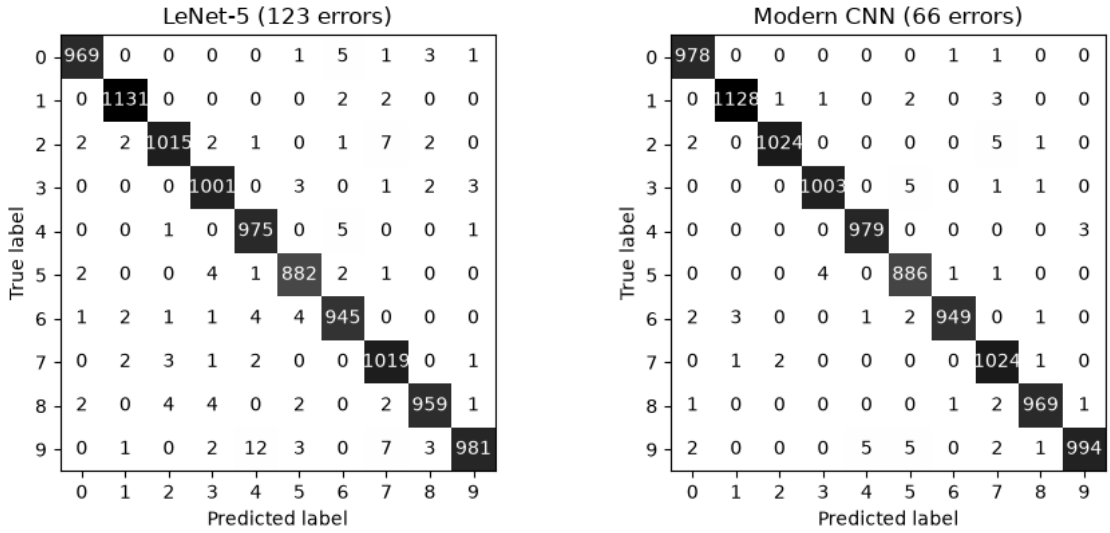
[36]:
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def predict(model, X, batch=1000):
    model.eval()
    with torch.no_grad():
        return torch.cat([model(X[i:i + batch]).argmax(dim=1) for i in range(0, len(X), batch)])

y_true = Ym_test.cpu().numpy()
fig, axes = plt.subplots(1, 2, figsize=(9.3, 4.2))
for ax, (model, name) in zip(axes, [(lenet, 'LeNet-5'), (cnn, 'Modern CNN')]):
    y_pred = predict(model, Xm_test).cpu().numpy()
    cm = confusion_matrix(y_true, y_pred)
    ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(ax=ax, cmap='Greys',␣
 ↪colorbar=False)
    errors = cm.sum() - cm.trace()
    ax.set_title(f'{name} ({errors} errors)')
    print(f'{name}: {errors} misclassified test images')

    # Most frequent mistake (largest off-diagonal entry)
    off = cm.copy()
    off[list(range(10)), list(range(10))] = 0
    i, j = divmod(off.argmax(), 10)
    print(f'  most common mistake: {i} predicted as {j} ({off[i, j]} times)')
plt.tight_layout()
plt.show()
```

```
LeNet-5: 123 misclassified test images
  most common mistake: 9 predicted as 4 (12 times)
Modern CNN: 66 misclassified test images
  most common mistake: 2 predicted as 7 (5 times)
```



Discussion: the modern CNN performed better. LeNet-5 reached 98.77% test accuracy (123 errors) and the modern CNN reached 99.34% (66 errors), so the CNN made about 46% fewer mistakes. It was also better from the first epoch (98.46% against 97.55%), because batch normalization and ReLU make training faster.

The curves show that LeNet-5 starts to overfit: from epoch 3 its training accuracy is higher than its test accuracy, and its test loss stops improving after epoch 7. For the CNN, the training accuracy stays below the test accuracy. This is because dropout is only active during training, which makes training harder. During testing, all units are used. So dropout stopped the CNN from overfitting, even though it has about four times more parameters (241,770 against 61,706).

In the confusion matrices, the mistakes are between digits that look similar. LeNet-5's most common mistake was 9 predicted as 4 (12 times), followed by 2 and 9 predicted as 7. The CNN fixed most of these, and none of its mistakes happened more than 5 times. The cost is speed: one CNN epoch took about 6.5 s, against about 5 s for LeNet-5.

Question 3

The hymenoptera dataset has only 244 training and 153 validation images of ants and bees. This is too small to train ResNet18 from scratch, so I used a ResNet18 pre-trained on ImageNet, following the PyTorch transfer learning tutorial.

Training images are randomly cropped and flipped (data augmentation), and validation images are only resized and center-cropped. All images are normalized with the ImageNet mean and standard deviation, which the pre-trained weights expect.

[37]:
```
import copy
import time
import torch.optim as optim
import numpy as np
from torchvision import datasets, models
from torchvision.models import ResNet18_Weights
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

imagenet_mean, imagenet_std = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(imagenet_mean, imagenet_std)]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(imagenet_mean, imagenet_std)]),
}

data_dir = './data/hymenoptera_data'
image_datasets = {x: datasets.ImageFolder(f'{data_dir}/{x}', data_transforms[x]) for x in␣
 ↪['train', 'val']}
dataloaders = {x: torch.utils.data.DataLoader(image_datasets[x], batch_size=4, shuffle=(x ==␣
 ↪'train'), num_workers=0)
               for x in ['train', 'val']}
dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes
print('Classes:', class_names)
print('Images:', dataset_sizes)
```

```
Classes: ['ants', 'bees']
Images: {'train': 244, 'val': 153}
```

Both parts use the same training function with the tutorial's settings: SGD with momentum 0.9, learning rate 0.001 (divided by 10 every 7 epochs), batch size 4 and 25 epochs. At the end, it keeps the weights from the epoch with the best validation accuracy.

[39]:
```
def train_model(model, optimizer, scheduler, num_epochs=25):
    criterion = nn.CrossEntropyLoss()
    history = {f'{phase}_{m}': [] for phase in ['train', 'val'] for m in ['loss', 'acc']}
    best_wts, best_acc = copy.deepcopy(model.state_dict()), 0.0
    start = time.time()

    for epoch in range(num_epochs):
        for phase in ['train', 'val']:
            model.train() if phase == 'train' else model.eval()
            running_loss, running_corrects = 0.0, 0
            for inputs, labels in dataloaders[phase]:
                inputs, labels = inputs.to(device), labels.to(device)
                optimizer.zero_grad()
                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)
                    if phase == 'train':
                        loss.backward()
                        optimizer.step()
                running_loss += loss.item() * inputs.size(0)
                running_corrects += (outputs.argmax(dim=1) == labels).sum().item()
            if phase == 'train':
                scheduler.step()
```

```
            history[f'{phase}_loss'].append(running_loss / dataset_sizes[phase])
            history[f'{phase}_acc'].append(100 * running_corrects / dataset_sizes[phase])
            if phase == 'val' and history['val_acc'][-1] > best_acc:
                best_acc = history['val_acc'][-1]
                best_wts = copy.deepcopy(model.state_dict())

        print(f"Epoch {epoch + 1}/{num_epochs}: "
              f"train loss {history['train_loss'][-1]:.4f}, acc {history['train_acc'][-1]:.2f}%␣
↪| "
              f"val loss {history['val_loss'][-1]:.4f}, acc {history['val_acc'][-1]:.2f}%")

    print(f'Training time: {time.time() - start:.0f}s, best validation accuracy: {best_acc:.
↪2f}%')
    model.load_state_dict(best_wts)
    return model, history, best_acc
```

(a) Fine-tuning

I replaced the last layer of ResNet18 (1000 ImageNet classes) with a new layer with 2 outputs (ants and bees), and trained the whole network. The small learning rate lets the pre-trained weights adjust slowly without losing what they learned from ImageNet.

[40]:
```
torch.manual_seed(0)
model_ft = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model_ft.fc = nn.Linear(model_ft.fc.in_features, len(class_names))
model_ft = model_ft.to(device)

trainable = sum(p.numel() for p in model_ft.parameters() if p.requires_grad)
print(f'Trainable parameters: {trainable:,}')

optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)
scheduler_ft = optim.lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)
model_ft, history_ft, best_acc_ft = train_model(model_ft, optimizer_ft, scheduler_ft,␣
↪num_epochs=25)
```

```
Trainable parameters: 11,177,538
Epoch 1/25: train loss 0.5849, acc 68.85% | val loss 0.2434, acc 91.50%
Epoch 2/25: train loss 0.5221, acc 77.05% | val loss 0.2342, acc 89.54%
Epoch 3/25: train loss 0.6879, acc 70.08% | val loss 0.3038, acc 89.54%
Epoch 4/25: train loss 0.4691, acc 80.74% | val loss 0.2478, acc 88.89%
Epoch 5/25: train loss 0.4900, acc 78.69% | val loss 0.2693, acc 88.89%
Epoch 6/25: train loss 0.2949, acc 88.52% | val loss 0.1666, acc 95.42%
Epoch 7/25: train loss 0.4321, acc 82.79% | val loss 0.2114, acc 90.85%
Epoch 8/25: train loss 0.3450, acc 84.84% | val loss 0.1801, acc 92.16%
Epoch 9/25: train loss 0.3598, acc 82.79% | val loss 0.2051, acc 92.16%
Epoch 10/25: train loss 0.3113, acc 86.48% | val loss 0.1734, acc 92.16%
Epoch 11/25: train loss 0.3733, acc 86.48% | val loss 0.1861, acc 92.16%
Epoch 12/25: train loss 0.2631, acc 87.30% | val loss 0.1628, acc 92.81%
Epoch 13/25: train loss 0.3338, acc 85.25% | val loss 0.1833, acc 92.16%
Epoch 14/25: train loss 0.2869, acc 86.48% | val loss 0.2103, acc 92.16%
Epoch 15/25: train loss 0.2765, acc 87.70% | val loss 0.2805, acc 88.89%
Epoch 16/25: train loss 0.2407, acc 88.52% | val loss 0.1842, acc 92.81%
Epoch 17/25: train loss 0.2522, acc 90.16% | val loss 0.2125, acc 92.16%
Epoch 18/25: train loss 0.3134, acc 85.25% | val loss 0.2355, acc 90.20%
Epoch 19/25: train loss 0.1736, acc 93.03% | val loss 0.1931, acc 92.16%
Epoch 20/25: train loss 0.2624, acc 88.11% | val loss 0.1824, acc 91.50%
Epoch 21/25: train loss 0.2929, acc 86.07% | val loss 0.1982, acc 91.50%
Epoch 22/25: train loss 0.2671, acc 88.93% | val loss 0.1728, acc 92.81%
```

```
Epoch 23/25: train loss 0.1931, acc 93.03% | val loss 0.1864, acc 92.81%
Epoch 24/25: train loss 0.3067, acc 84.43% | val loss 0.1754, acc 91.50%
Epoch 25/25: train loss 0.2290, acc 92.62% | val loss 0.1820, acc 92.16%
Training time: 111s, best validation accuracy: 95.42%
```

(b) Using the network as a feature extractor

Here I froze all the pre-trained layers (`requires_grad = False`), so they keep their ImageNet weights, and trained only the new 2-output layer. Everything else is the same as in part (a).

[41]:
```
torch.manual_seed(0)
model_fe = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
for param in model_fe.parameters():
    param.requires_grad = False  # freeze the pre-trained layers

model_fe.fc = nn.Linear(model_fe.fc.in_features, len(class_names))  # new layer, trainable
model_fe = model_fe.to(device)

trainable = sum(p.numel() for p in model_fe.parameters() if p.requires_grad)
print(f'Trainable parameters: {trainable:,}')

optimizer_fe = optim.SGD(model_fe.fc.parameters(), lr=0.001, momentum=0.9)
scheduler_fe = optim.lr_scheduler.StepLR(optimizer_fe, step_size=7, gamma=0.1)
model_fe, history_fe, best_acc_fe = train_model(model_fe, optimizer_fe, scheduler_fe,␣
 ↪num_epochs=25)
```

```
Trainable parameters: 1,026
Epoch 1/25: train loss 0.6504, acc 63.93% | val loss 0.2278, acc 90.85%
Epoch 2/25: train loss 0.5138, acc 75.82% | val loss 0.1872, acc 95.42%
Epoch 3/25: train loss 0.6806, acc 70.08% | val loss 0.1718, acc 94.77%
Epoch 4/25: train loss 0.4656, acc 81.97% | val loss 0.1525, acc 96.08%
Epoch 5/25: train loss 0.4344, acc 79.92% | val loss 0.2104, acc 92.81%
Epoch 6/25: train loss 0.3057, acc 85.25% | val loss 0.1705, acc 95.42%
Epoch 7/25: train loss 0.3711, acc 82.38% | val loss 0.1670, acc 95.42%
Epoch 8/25: train loss 0.3751, acc 84.43% | val loss 0.1589, acc 95.42%
Epoch 9/25: train loss 0.4059, acc 80.33% | val loss 0.1724, acc 94.77%
Epoch 10/25: train loss 0.3722, acc 85.25% | val loss 0.1539, acc 95.42%
Epoch 11/25: train loss 0.4110, acc 81.97% | val loss 0.1569, acc 94.12%
Epoch 12/25: train loss 0.3158, acc 88.11% | val loss 0.1595, acc 95.42%
Epoch 13/25: train loss 0.3722, acc 82.38% | val loss 0.1547, acc 95.42%
Epoch 14/25: train loss 0.3270, acc 82.79% | val loss 0.1867, acc 92.81%
Epoch 15/25: train loss 0.2991, acc 87.30% | val loss 0.2152, acc 92.81%
Epoch 16/25: train loss 0.3018, acc 86.48% | val loss 0.1643, acc 94.77%
Epoch 17/25: train loss 0.2955, acc 86.07% | val loss 0.1712, acc 94.77%
Epoch 18/25: train loss 0.3937, acc 81.97% | val loss 0.2020, acc 92.81%
Epoch 19/25: train loss 0.2339, acc 89.75% | val loss 0.1604, acc 94.77%
Epoch 20/25: train loss 0.3071, acc 87.70% | val loss 0.1643, acc 95.42%
Epoch 21/25: train loss 0.3672, acc 84.02% | val loss 0.1763, acc 94.77%
Epoch 22/25: train loss 0.3269, acc 83.61% | val loss 0.1587, acc 96.08%
Epoch 23/25: train loss 0.2447, acc 90.16% | val loss 0.1721, acc 94.77%
Epoch 24/25: train loss 0.3394, acc 83.20% | val loss 0.1588, acc 94.12%
Epoch 25/25: train loss 0.3143, acc 88.11% | val loss 0.1695, acc 94.77%
Training time: 101s, best validation accuracy: 96.08%
```

Comparing the two approaches

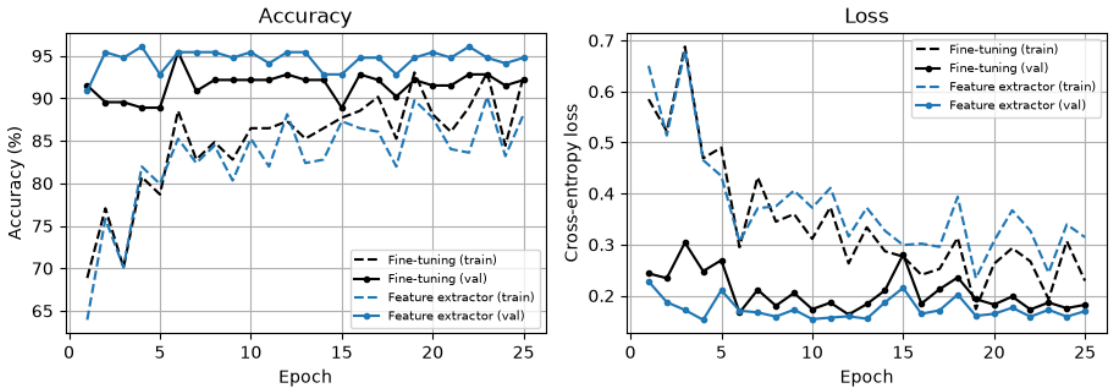
Accuracy and loss per epoch for both approaches (dashed = training, solid = validation).

[42]:
```
epochs = range(1, 26)
runs = [(history_ft, 'Fine-tuning', 'k'), (history_fe, 'Feature extractor', 'tab:blue')]

fig, axes = plt.subplots(1, 2, figsize=(9.3, 3.4))
for hist, name, color in runs:
    axes[0].plot(epochs, hist['train_acc'], '--', color=color, label=f'{name} (train)')
    axes[0].plot(epochs, hist['val_acc'], '-o', color=color, markersize=3, label=f'{name} (val)')
    axes[1].plot(epochs, hist['train_loss'], '--', color=color, label=f'{name} (train)')
    axes[1].plot(epochs, hist['val_loss'], '-o', color=color, markersize=3, label=f'{name}␣
 ↪(val)')

axes[0].set_title('Accuracy')
axes[0].set_ylabel('Accuracy (%)')
axes[1].set_title('Loss')
axes[1].set_ylabel('Cross-entropy loss')
for ax in axes:
    ax.set_xlabel('Epoch')
    ax.grid(True)
    ax.legend(fontsize=7)
plt.tight_layout()
plt.show()

for hist, name, _ in runs:
    print(f"{name:<18} best val acc {max(hist['val_acc']):.2f}%, final val acc␣
 ↪{hist['val_acc'][-1]:.2f}%, "
          f"mean val acc over the last 10 epochs {np.mean(hist['val_acc'][-10:]):.2f}%")
```




```
Fine-tuning        best val acc 95.42%, final val acc 92.16%, mean val acc over
the last 10 epochs 91.96%
Feature extractor  best val acc 96.08%, final val acc 94.77%, mean val acc over
the last 10 epochs 94.71%
```

Next, I used both models to classify all 153 validation images and plotted the images that at least one model got wrong.

[43]:
```
def show_tensor(ax, img, title):
    # Undo the normalization so the image can be displayed
    img = img.numpy().transpose((1, 2, 0)) * np.array(imagenet_std) + np.array(imagenet_mean)
    ax.imshow(np.clip(img, 0, 1))
    ax.set_title(title)
```

```
    ax.axis('off')

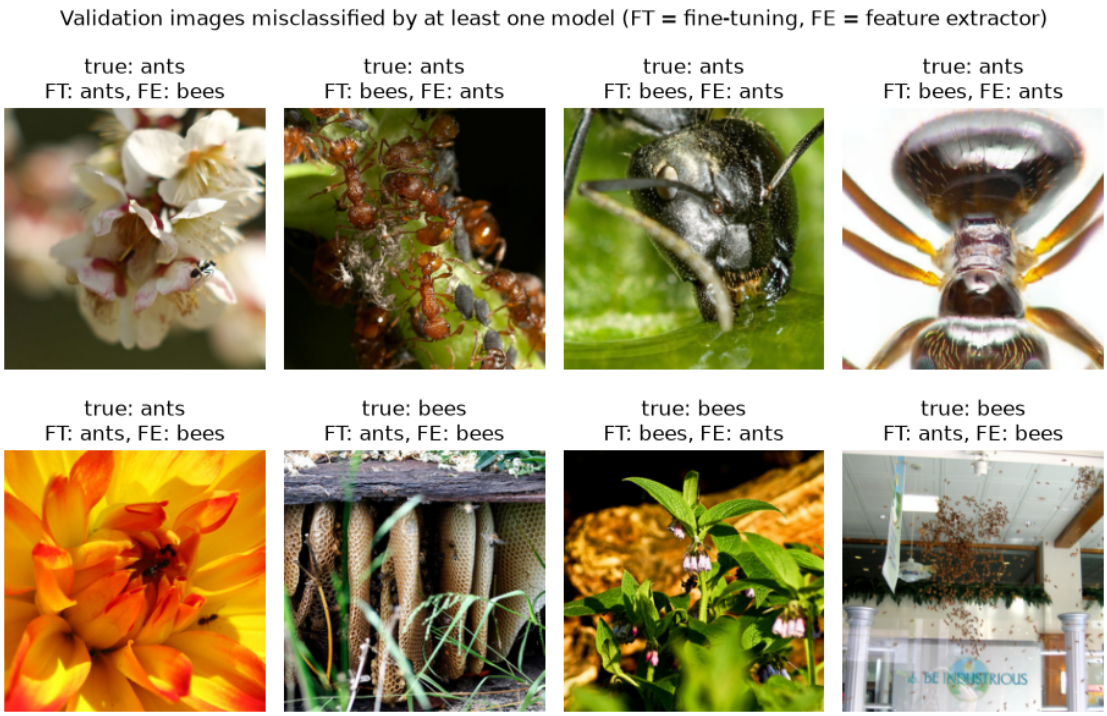
def predict_all(model, dataset):
    model.eval()
    loader = torch.utils.data.DataLoader(dataset, batch_size=32, shuffle=False)
    preds = []
    with torch.no_grad():
        for inputs, _ in loader:
            preds.append(model(inputs.to(device)).argmax(dim=1).cpu())
    return torch.cat(preds)

val_set = image_datasets['val']
y_val = torch.tensor(val_set.targets)
pred_ft = predict_all(model_ft, val_set)
pred_fe = predict_all(model_fe, val_set)

wrong_ft = set((pred_ft != y_val).nonzero().flatten().tolist())
wrong_fe = set((pred_fe != y_val).nonzero().flatten().tolist())
print(f'Fine-tuning: {len(wrong_ft)} errors, accuracy {100 * (1 - len(wrong_ft) / len(y_val)):.
 ↪2f}%')
print(f'Feature extractor: {len(wrong_fe)} errors, accuracy {100 * (1 - len(wrong_fe) /␣
 ↪len(y_val)):.2f}%')
print(f'Misclassified by both models: {len(wrong_ft & wrong_fe)}')

shown = sorted(wrong_ft | wrong_fe)[:8]
fig, axes = plt.subplots(2, 4, figsize=(9.3, 6.4))
for ax in axes.flat:
    ax.axis('off')
for ax, i in zip(axes.flat, shown):
    show_tensor(ax, val_set[i][0], f'true: {class_names[y_val[i]]}\n'
                f'FT: {class_names[pred_ft[i]]}, FE: {class_names[pred_fe[i]]}')
plt.suptitle('Validation images misclassified by at least one model (FT = fine-tuning, FE =␣
 ↪feature extractor)')
plt.tight_layout()
plt.show()
```

```
Fine-tuning: 7 errors, accuracy 95.42%
Feature extractor: 6 errors, accuracy 96.08%
Misclassified by both models: 1
```



Discussion: both approaches worked well, because the ImageNet features are already useful for ants and bees: both models were above 90% after just one epoch. The feature extractor reached 96.08% (6 wrong) and fine-tuning reached 95.42% (7 wrong), even though the feature extractor trains only 1,026 parameters and fine-tuning trains over 11 million.

The feature extractor was also more stable. Its validation accuracy stayed between about 93% and 96% (average 94.71% over the last 10 epochs), while fine-tuning dropped to 90-93% after its best epoch (average 91.96%). With only 244 training images, changing 11 million weights makes the model fit the training images too closely. Since this dataset is small and similar to ImageNet, the frozen features are enough. Fine-tuning would be better with more data or images that are very different from ImageNet.

The training accuracy is lower than the validation accuracy because the training images are randomly cropped and flipped, which makes them harder. Training time was similar (111 s and 101 s), because most of the time is spent loading the images.

The two models made different mistakes, and only one image was wrong for both. Most mistakes are images where the insect is very small, where no clear insect is visible (honeycomb, a swarm of bees, a plant), or unusual close-ups. In these images the background, such as flowers, has a big effect on the prediction.